

Relay: An Outbound Message Hub

Object-Oriented Programming — Standalone Lab

Marks: 10 **Language:** Java (JDK 17) **Approach:** Test-Driven

Submit your .java source files and do not modify the test file.

The job

A company sends a lot of outbound notices — order confirmations, one-time codes, delivery alerts. Those notices leave over different *channels*: some by e-mail, some by SMS, some as a phone push. The same notice text looks different on each channel, costs a different amount to send, and reads differently in the audit log. Your job is to build the small system that holds a batch of these outbound notices and sends the whole batch in one go, reporting back a full transmission log and the total cost of the batch.

Start by looking at a version that already failed

A previous intern built this. Their design was one class, `Notice`, with a `String channel` field and a method that branched on it:

```
class Notice {
    String channel;          // "EMAIL", "SMS", or "PUSH"
    String body;
    // ... whoAmI getters ...

    transmit() {
        if (channel == "EMAIL")    // build an e-mail line
        else if (channel == "SMS") // split into segments, count them
        else if (channel == "PUSH") // truncate for the lock screen
    }
    cost() {
        if (channel == "EMAIL")    // free
        else if (channel == "SMS") // per-segment charge
        else if (channel == "PUSH") // flat
    }
}
```

It worked for a week. Then these things happened, and each one hurt:

The intern's version, and why it hurts

- **Adding a channel meant editing every method.** A new “WhatsApp” channel forced a new branch into `transmit`, another into `cost`, another into the audit-log method — the *same* list of channels, copied and re-typed in three places, drifting out of sync the moment someone forgot one.
- **The branch ladders were everywhere.** Anywhere the code asked “what should this notice do?” it first had to ask “what *kind* of notice is this?” The `if/else` on channel reappeared in every method that did anything interesting.
- **A half-built notice was a legal object.** `new Notice()` with no channel set compiled fine and then exploded at send time, because none of the branches matched.
- **“Priority e-mail” broke it completely.** Marketing asked for a priority e-mail: it sends *exactly* like an e-mail but with a flag in front and a small surcharge. In the intern’s design

there was nowhere to say “the same as e-mail, plus a little” — they ended up copying the entire e-mail branch and pasting it into a new “PRIORITY_EMAIL” branch, so now the e-mail logic lived in two places.

You are rebuilding this from scratch. **Nothing below tells you which Java features to use.** It tells you how the world behaves; the shape of your classes is the thing you are being marked on. But take the four pains above as a promise: if your design has the same shape as the intern’s, it will hit the same four walls — and the test file will catch it at the first one.

How the world behaves

Read these as facts about the business. Each one quietly decides something about how your classes must relate. The italic phrases are the load-bearing ones.

The notices

- Every notice carries the same core: *who* it is going to and the *text* to send. That much is true of all of them, written down *once*, no matter the channel.
- Beyond that core, the channels differ in three behaviours — how a notice is *put on the wire*, what it *costs* to send, and how it *reads in the audit log*:
 - **E-mail** renders the full text with a header naming the recipient; it is *free* to send.
 - **SMS** is cut into 160-character segments; it states how many segments it became, and it is charged *per segment* (so a longer text genuinely costs more than a short one).
 - **Push** is trimmed to a short lock-screen preview; it is charged a small *flat* amount.
- There is no such thing as a “plain notice” with no channel. *A notice that does not belong to a specific channel cannot be put on the wire, cannot be priced, and should never come into existence in the first place.*

The priority e-mail

- A *priority e-mail* is not a new channel. It is an e-mail — it goes out exactly the way an e-mail goes out — with a priority flag added *in front of* whatever the e-mail would already have produced, and a small express surcharge added *on top of* whatever the e-mail would already have cost.
- This is the rule the intern could not express: if the ordinary e-mail behaviour is ever changed, the priority e-mail must change *with it, for free*, without anyone re-typing the e-mail logic. The priority e-mail must *build on* the e-mail, not copy it.

The recipient

- A recipient is a small thing of its own — a name and an address — that a notice *has*. A notice is plainly not a kind of recipient, and a recipient is not a kind of notice; one simply *holds* the other and asks it for the name and address when it is time to go out.

The hub (the “outbox”)

- The outbox holds a batch of outbound notices — e-mails, texts, pushes, priority e-mails, all together in *one* place. From the outbox’s point of view they are simply “things to be sent,” and it must be able to *hold any of them side by side* without caring which channel each one is.

- **Send the batch in one sweep.** Flushing the outbox transmits *everything* it holds in a single pass — “for each thing waiting to go out, put it on the wire” — and hands back the combined log. That loop must not contain a ladder that asks each item what channel it is; each item must already *know* how to put itself on the wire.
- **Total the batch in one sweep.** The outbox can report what the whole batch costs to send, again by walking the queue once and asking each thing what it costs — with no per-channel branching in the walk.
- **One name, two ways to queue.** Queuing a notice should be possible either on its own (send it once) or with a repeat count (send the same notice several times). Same idea, same name, different inputs — the caller should not have to remember two different method names for “put this in the outbox.”
- The outbox can also say how many notices are currently waiting.

Decide the relationships before you type. Every italic phrase above — “written down once,” “should never come into existence,” “is an e-mail ... added on top of,” “a notice *has* a recipient,” “hold any of them side by side,” “each item must already know” — is describing a relationship between two of your classes. Work out, on paper, which classes those are and how they connect, *before* you open the editor. Students who start by typing classes almost always discover halfway through that two of them should have been one, or that one of them should have been built on another.

A Java tool you will need: cutting a piece out of a String

The push channel has to “trim” its text down to a short preview. To do that you need two String operations you may not have met yet. Neither has anything to do with the design of this lab; they are just the mechanics of working with text, so we explain them up front.

length() — how many characters a String holds. Calling `.length()` on a String gives you its size as an int. It counts every character, including spaces.

```
String s = "hello";
int n = s.length();           // n is 5
"".length();                  // 0 (an empty String)
"good day".length();          // 8 (the space counts)
```

substring(...) — copy out a slice of a String. `substring` hands you back a *new* String containing part of the original; it never changes the original. The characters of a String are numbered from 0, so in “hello” the ‘h’ is at position 0, the ‘e’ at 1, and so on. There are two ways to call it:

```
String s = "Notification";
//      0123456789...      <- positions (start counting at 0)

// One argument: from that position to the end of the String.
s.substring(4);             // "fication" (everything from position 4 onward)

// Two arguments: from the first position UP TO (but NOT including) the
// second.
s.substring(0, 4);           // "Noti"      (positions 0,1,2,3 -- not 4)
s.substring(2, 6);           // "tifi"      (positions 2,3,4,5)
```

A useful way to read `s.substring(0, k)` is “the first k characters of `s`.” So `s.substring(0, 40)` is the first 40 characters — exactly the kind of slice a preview needs.

▷ Notice

Two traps. (1) The second argument is the position to stop *before*, not the number of characters after the first — `substring(0, 4)` gives four characters (0, 1, 2, 3), not five. (2) Asking for a slice that runs past the end of the String — e.g. `"hi".substring(0, 40)` — throws an error at run time. So before you cut a String down to its first 40 characters, you must first check with `length()` whether it is even that long; a text shorter than the preview size needs no trimming at all.

Step 0 — Sketch the interactions (do this first, and submit it)

Before any code, draw — by hand is fine — how these objects hand work to one another when a batch is flushed. Show, for example: who the outbox talks to when you flush it; what an e-mail notice has to ask before it can produce its line; what the priority e-mail leans on to do its job; where the recipient comes in. Use boxes and arrows and your own labels; **this is not UML and needs no formal notation** — a clear informal picture is exactly what we want. Photograph or scan it and submit it with your code. You will be asked to talk through it.

Question Your Viva Will Ask (the sketch)

Point at the single arrow in your drawing that does the real work when the outbox is flushed — the one call that, repeated over the queue, somehow produces an e-mail line for one item and a segmented SMS line for the next. Who decides which of those two things actually happens, and at what instant: when you wrote the loop, or while the loop is running? If your drawing has a separate arrow per channel, you have drawn the intern’s version again.

What each class must expose

Here is the catalogue of behaviours your classes must offer, by *name and effect only*. Notice what is deliberately missing: we are **not** telling you any method’s parameters or its return type, and we are not telling you which class should sit above which. You have done test-driven development before; the provided test file is your specification for the exact signatures, and the relationships are yours to design from the rules above.

- **A recipient** can be asked for its name and its address.
- **Every notice**, whatever its channel, can:
 - *deliver* itself — produce the exact text that would go out on its channel;
 - report its *cost* to send;
 - *describe* itself — a one-line summary suitable for an audit log.

A channel-less notice can answer *none* of these, and must not be constructible.

- **An e-mail notice, an SMS notice, and a push notice** each provide the three behaviours above in their own way (per the channel rules).
- **A priority e-mail notice** provides them too — but its delivered text and its cost are built *from* the ordinary e-mail’s, not written afresh.
- **The outbox** can: take a notice to be sent (and, by the same name, take a notice plus a repeat count); *flush* the whole batch and return the combined transmission log; report the batch’s *total cost*; and report how many notices are waiting.

▷ Notice

The test file constructs each kind of notice *by its own name* and then drops a mixture of them into a single outbox. A design that funnels every channel through one class with a “which channel am I” field will not even compile against the test, let alone pass it — and that, precisely, is the lesson. Read the test file the way you have been taught: let it tell you each method’s return type and parameters; let *this* document tell you how the classes relate.

What the test file checks (in words — read the file for the types)

You will be given `RelayTest.java`. You do not modify it. The following is what it verifies, described so you know what to build; the exact signatures are in the file.

- **Each channel speaks in its own voice.** Build one notice of each channel to the same recipient and the same text; check that the e-mail’s delivered text carries the recipient header, the SMS’s states a segment count, and the push’s is the trimmed preview — three visibly different results from one request.
- **Cost follows behaviour, not a stored number.** A long SMS text comes out costing more than a short one because it splits into more segments; an e-mail comes out free; a push comes out at the flat amount. (None of these is a value you simply stored and handed back.)
- **The priority e-mail is built on the e-mail.** For the same recipient and text, the priority e-mail’s delivered text *contains, verbatim*, the line a plain e-mail would have produced, with the priority flag ahead of it; and its cost equals the plain e-mail’s cost plus the surcharge. A later check changes what the e-mail produces and confirms the priority e-mail moves with it, with no edit of its own.
- **One outbox, many kinds, side by side.** Drop an e-mail, an SMS, a push, and a priority e-mail into one outbox; it reports four waiting and a total cost equal to the four individual costs added up by hand.
- **Flushing sends everything in one sweep.** The log returned by a flush contains every queued notice’s own delivered text, in the order they were queued.
- **One slot, many forms.** A single variable is pointed at an e-mail, then re-pointed at an SMS, then at a push; each time it is queued and later flushes correctly — the same variable standing in for any channel.
- **Queue once, or queue many.** Queuing a notice with a repeat of three leaves three waiting and triples that notice’s share of the total; queuing it the plain way leaves exactly one.

Question Your Viva Will Ask (the total)

Your “total cost” walks the queue and asks each item the *same* question under the *same* name, yet a short SMS, a long SMS, a free e-mail, and a priority e-mail all return different numbers. The walk was written before any particular batch existed. *Who* picks the right cost rule for each item, and *when*? Contrast that with the two ways of queuing a notice (plain, or with a repeat): which of those two is chosen by the same mechanism, and which is chosen earlier, by the compiler, from the way the call was written?

Marking Rubric

#	Criterion	Marks
1	The four notice kinds each deliver, price, and describe themselves in their own way, and can be held together in one outbox through a single common type	3
2	Flush and total each work as a single sweep over the mixed queue, with no per-channel branch ladder inside the sweep	2
3	The priority e-mail is built <i>on</i> the e-mail (its delivery and cost reuse the e-mail's rather than copying them) and sits correctly in the chain above it	2
4	The shared notice core (recipient, text) is written once and is sealed against outside tampering; a channel-less notice cannot be constructed	1.5
5	Queuing offers one name with two input shapes (plain, and with a repeat count)	1
6	Recipient modelled as something a notice <i>holds</i> , not a kind of notice; clear informal interaction sketch submitted	0.5
Total		10

▷ Notice

Passing the tests is necessary but not sufficient. Two properties cannot be written as an assertion — that a channel-less notice is impossible to construct, and that the shared core is sealed against outside code — and the is-a / has-a choices are a matter of design, not output. Criteria 3, 4, and 6 are judged by reading your code and by the viva. A solution that reaches the right numbers through one class and a forest of `if/else` on a channel field has rebuilt the intern's version and will score accordingly, however green the test bar looks.

Ground rules

- Java 17. Use `double` for money and `int` for counts and lengths.
- Fixed-size arrays only, each tracked by its own `int` count. Do *not* import `java.util.ArrayList`, `List`, `HashMap`, or any other collection or helper class — the outbox's storage is a plain array, and that is on purpose.
- Say every class relationship aloud before you commit to it: “*is a*” versus “*has a*.” There is exactly one “has-a” pairing in this lab and the rest are “is-a”; getting that split right is most of the design.
- Partial credit is awarded; submit whatever compiles. Submit your source files and your Step-0 sketch.